# DECA Lab Logbook

*Part 1 | Section 2: Digital Arithmetic*

## Section 2: Digital Arithmetic

### Aim

To build arithmetic circuits from the ground up in a new Issie project 'arithmetic': half adder → full adder → 4-bit ripple-carry adder → controllable inverter → adder/subtractor → expanded ALU with both bitwise and arithmetic operations.

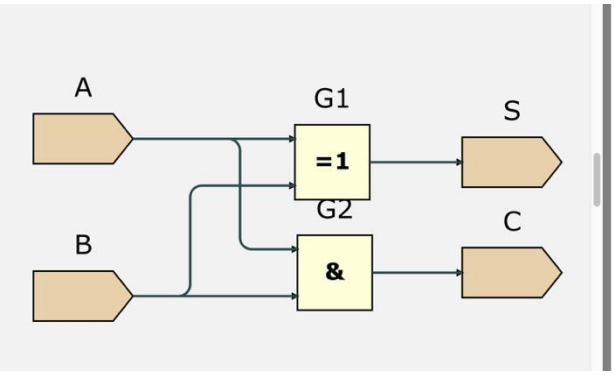
### Half Adder

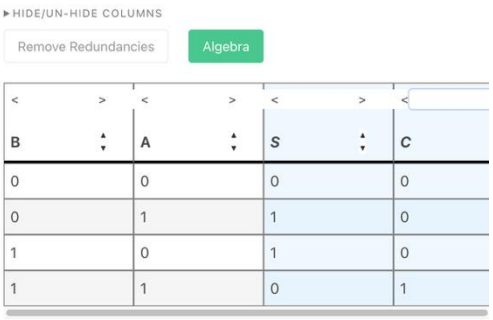
#### *Circuit Description*

Adds two 1-bit inputs A and B, producing a sum S and carry-out C. The Boolean functions are:

$S = A \oplus B$ *(XOR gate)*

$C = A \cdot B$ *(AND gate)*




| B | A | S | C |
|---|---|---|---|
| 0 | 0 | 0 | 0 |
| 0 | 1 | 1 | 0 |
| 1 | 0 | 1 | 0 |
| 1 | 1 | 0 | 1 |

#### *How It Works*

- Sum: XOR gate — two identical inputs give 0; different inputs give 1
- Carry out: AND gate — only two 1s generate a carry of 1

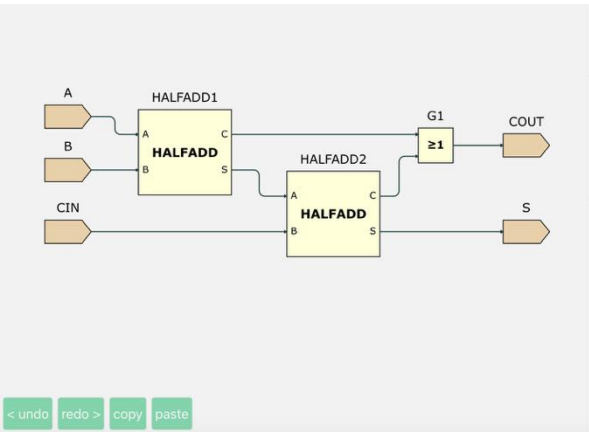
#### *Truth Table*

| A | B | C (carry) | S (sum) |
|---|---|---|---|
| 0 | 0 | 0 | 0 |
| 0 | 1 | 0 | 1 |
| 1 | 0 | 0 | 1 |
| 1 | 1 | 1 | 0 |

### Full Adder

#### *Circuit Description*

Adds three 1-bit inputs A, B and CIN, producing sum S and carry-out COUT. Built from two instances of HALFADD and one OR gate.




| CIN | A | B | COUT | S |
|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 0 |
| 0 | 0 | 1 | 0 | 1 |
| 0 | 1 | 0 | 0 | 1 |
| 0 | 1 | 1 | 1 | 0 |
| 1 | 0 | 0 | 0 | 1 |
| 1 | 0 | 1 | 1 | 0 |
| 1 | 1 | 0 | 1 | 0 |
| 1 | 1 | 1 | 1 | 1 |

### *How It Works*

The first HALFADD computes the partial sum and carry of A and B. The second HALFADD adds the partial sum with CIN. The final carry-out is the OR of the two half-adder carries — at most one will be 1 at a time.

### *Truth Table (exhaustive — 8 combinations)*

| CIN | A | B | COUT | S |
|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 0 |
| 0 | 0 | 1 | 0 | 1 |
| 0 | 1 | 0 | 0 | 1 |
| 0 | 1 | 1 | 1 | 0 |
| 1 | 0 | 0 | 0 | 1 |
| 1 | 0 | 1 | 1 | 0 |
| 1 | 1 | 0 | 1 | 0 |
| 1 | 1 | 1 | 1 | 1 |

## Ripple-Carry Adder (FA4)

### *Circuit Description*

A 4-bit adder built by chaining four FULLADD instances. The carry output of bit n feeds into the carry input of bit n+1, mimicking pencil-and-paper column addition.

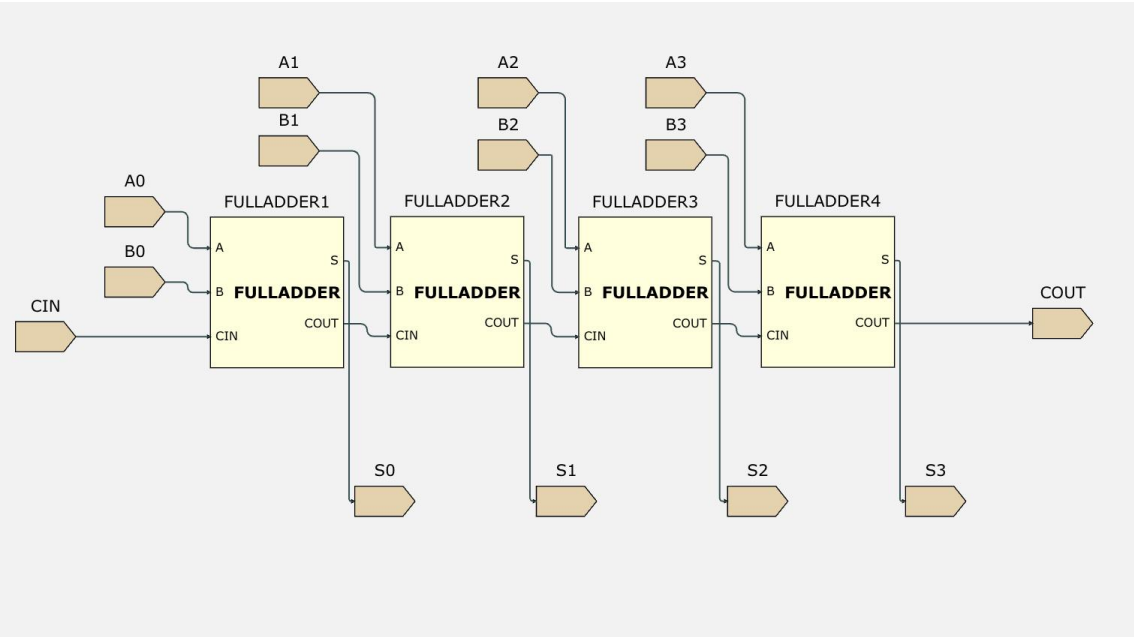
### *Components*

- 4-bit inputs A(3:0) and B(3:0), carry input CIN
- 4-bit sum output S(3:0) and carry-out COUT
- Four FULLADD instances, one per bit position

### *How It Works*

Each full adder processes one bit of A and B plus the carry from the previous stage. CIN of bit 0 is tied to 0; COUT of bit 3 is the overall carry-out. Simulated with random A, B inputs to confirm S and COUT are correct.

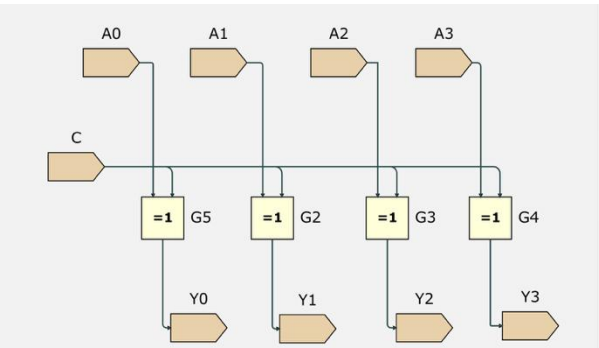
The circuit diagram is in the next page.

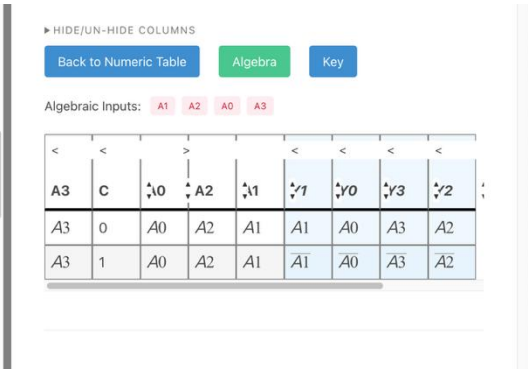



## Controllable Inverter (CINV)

### *Circuit Description*

A 4-bit block that either passes its input unchanged or inverts all bits, based on a 1-bit control C. Implemented using four XOR gates (one per bit).




| A3 | C | A0 | A2 | A1 | Y1 | Y0 | Y3 | Y2 |
|---|---|---|---|---|---|---|---|---|
| A3 | 0 | A0 | A2 | A1 | A1 | A0 | A3 | A2 |
| A3 | 1 | A0 | A2 | A1 | $\overline{A1}$ | $\overline{A0}$ | $\overline{A3}$ | $\overline{A2}$ |

### *How It Works*

XOR property: $A \oplus 0 = A$ (pass through) and $A \oplus 1 = \bar{A}$ (invert). Connecting control C to one input of each XOR gate enables or disables inversion for all four bits simultaneously.

### *Algebraic Truth Table*

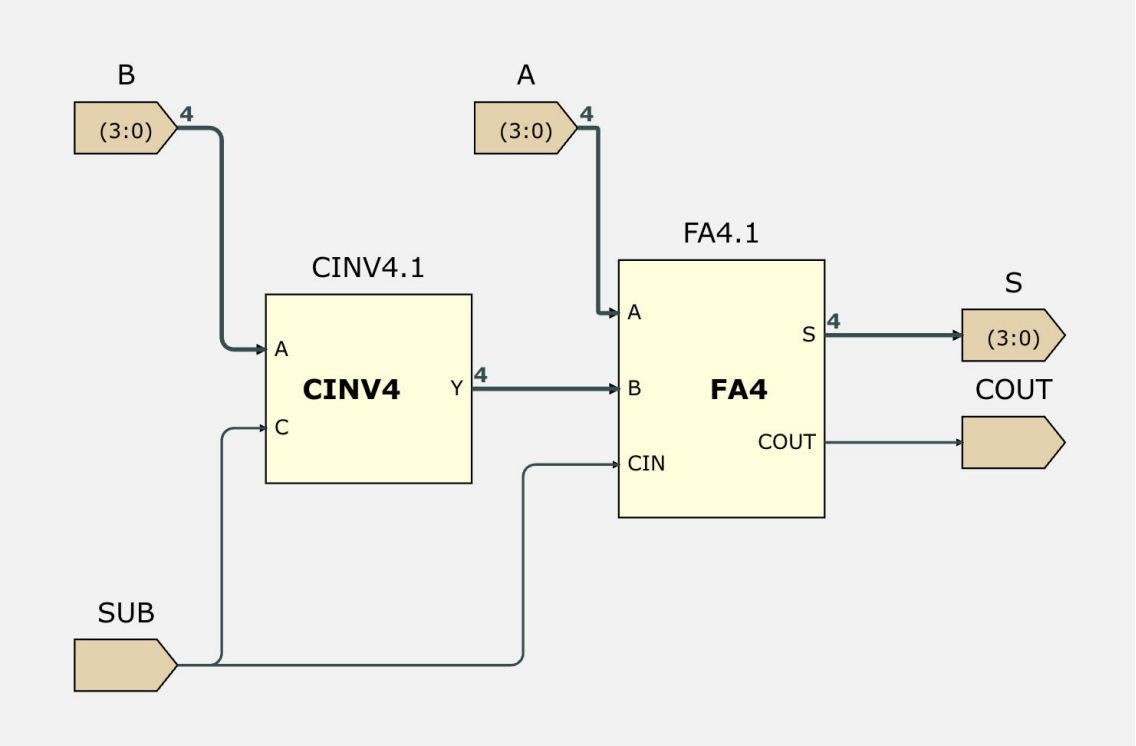
| C | Y0 | Y1 | Y2 | Y3 |
|---|---|---|---|---|
| 0 | A0 | A1 | A2 | A3 (unchanged) |
| 1 | $\bar{A}0$ | $\bar{A}1$ | $\bar{A}2$ | $\bar{A}3$ (inverted) |

## 4-bit Adder/Subtractor (ADDSUB)

### *Circuit Description*

Combines CINV and FA4 to create a block that adds or subtracts based on the single control signal

SUB. Two's complement negation requires bitwise inversion plus adding 1 — both are enabled by the same SUB signal.




### *How It Works*

When SUB = 0: CINV passes B unchanged and CIN = 0, so FA4 computes A + B.

When SUB = 1: CINV inverts B and SUB also drives CIN = 1, so FA4 computes A + $\overline{B}$ + 1 = A − B (two's complement subtraction).

### *Operation Table*

| SUB | Operation | CIN to FA4 | B input to FA4 |
|---|---|---|---|
| 0 | Addition (A + B) | 0 | B (unchanged) |
| 1 | Subtraction (A − B) | 1 | $\overline{B}$ (inverted) |

Simulated with random A, B values for both SUB = 0 and SUB = 1. Results confirmed correct for all test cases.

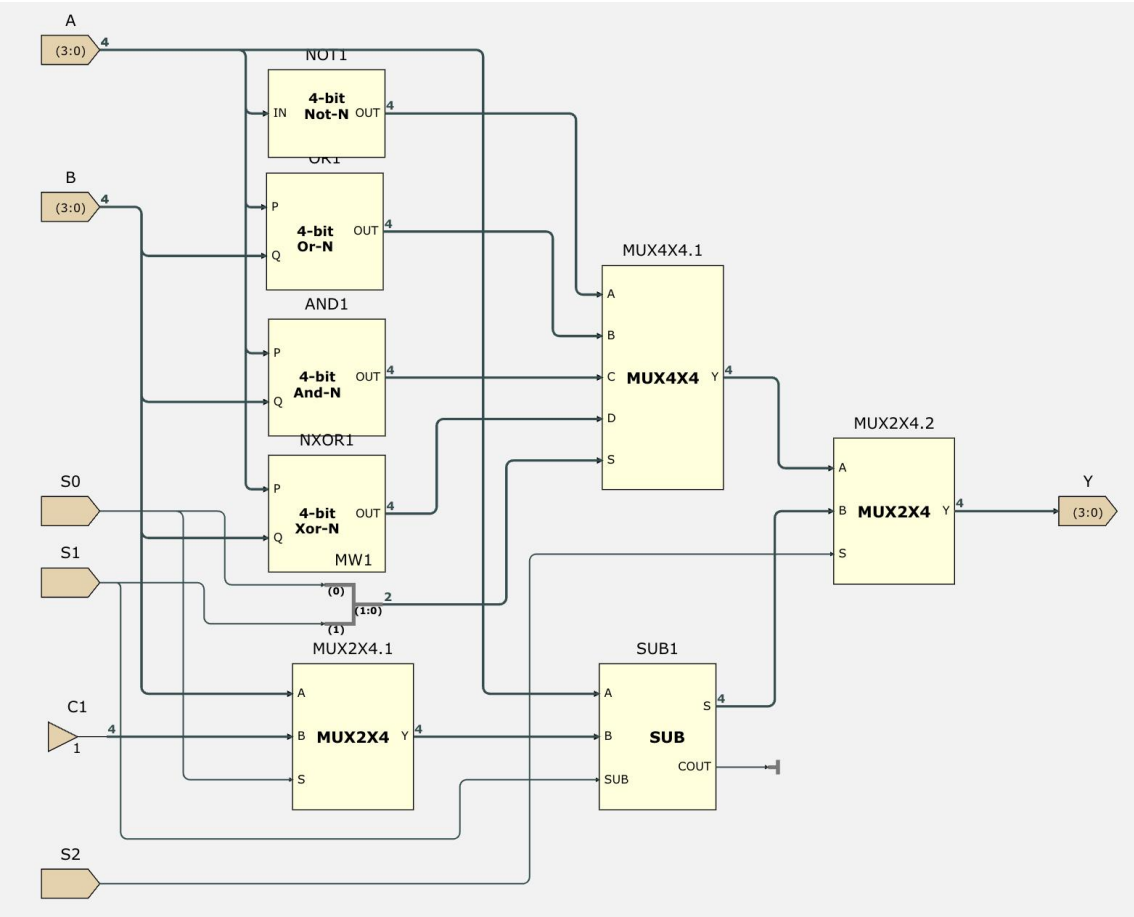
## Challenge: Expanded ALU with Arithmetic Operations

### *Aim*

Extend the Section 1 bitwise ALU to include four arithmetic operations, giving 8 total functions controlled by a 3-bit instruction code S(2:0).

### *Additional Components*

- One ADDSUB block
- MUX2X4 to select between B and constant 1 as the second operand
- MUX2X4 to select between the bitwise result and the arithmetic result
- 3-bit select input S(2:0); S2 selects bitwise vs arithmetic




### *How It Works*

S2 = 0 routes to the bitwise unit (from Section 1). S2 = 1 routes to the arithmetic unit. Within the arithmetic block: S0 selects the second operand (B or constant 1), and S1 controls add vs. subtract in the ADDSUB block.

### *Instruction Table*

| S2 | S1 | S0 | Operation |
|---|---|---|---|
| 0 | 0 | 0 | Bitwise NOT A |
| 0 | 0 | 1 | A OR B |
| 0 | 1 | 0 | A AND B |
| 0 | 1 | 1 | A XOR B |
| 1 | 0 | 0 | A + B |
| 1 | 0 | 1 | A + 1 |
| 1 | 1 | 0 | A − B |
| 1 | 1 | 1 | A − 1 |

All 8 instruction codes verified by simulation with random values of A and B. Results match the expected operation in every case.